

Solving the 15-Puzzle with Weighted A* Search

2205117

July 6, 2026

1 Introduction

This report describes an experiment in which the Weighted A* search algorithm was applied to the 15-puzzle. A single admissible heuristic, the Linear Conflict heuristic, was combined with four different heuristic weights in order to study the trade-off between solution quality and search effort. Twelve distinct solvable 4×4 puzzle instances were used for the evaluation.

2 Implementation Summary

The solver was built around the following components:

- A 4×4 puzzle representation and move generator
- An inversion-parity check to confirm each instance is solvable before search begins
- A priority-queue-based Weighted A* search routine
- The Linear Conflict heuristic
- A benchmarking harness that repeats every puzzle across a set of weights

As in standard A*, nodes are ordered by the evaluation function

$$f(n) = g(n) + W \times h(n), \tag{1}$$

where $g(n)$ is the cost of the path taken so far, $h(n)$ is the heuristic estimate of the remaining distance to the goal, and $W \geq 1$ is the heuristic weight. Setting $W = 1$ recovers ordinary A*; larger values of W make the search increasingly greedy, trading the guarantee of optimality for a smaller search tree.

3 Experimental Setup

Twelve scrambled, solvable 4×4 puzzles were generated. Each puzzle was solved four separate times, once for every weight in

$$W \in \{1.0, 1.2, 2.0, 5.0\}. \tag{2}$$

For every run, two metrics were recorded:

- Solution cost – the number of moves in the returned solution

- Expanded nodes – the total number of nodes popped from the priority queue during the search

All 48 runs (12 puzzles $\times$ 4 weights) used the Linear Conflict heuristic exclusively, so any change in behaviour across a row of the table below is attributable solely to the weight.

4 Results

Table 1 reports the solution cost and number of expanded nodes for every puzzle/weight combination.

Puzzle	Weight	Solution Cost	Expanded Nodes
1	1.0	29	2,741
1	1.2	29	772
1	2.0	29	130
1	5.0	41	107
2	1.0	27	102
2	1.2	27	43
2	2.0	27	39
2	5.0	35	72
3	1.0	34	17,625
3	1.2	34	4,658
3	2.0	40	743
3	5.0	66	619
4	1.0	32	46,580
4	1.2	32	18,790
4	2.0	32	8,987
4	5.0	44	1,089
5	1.0	30	2,190
5	1.2	30	755
5	2.0	32	261
5	5.0	44	617
6	1.0	19	24
6	1.2	19	20
6	2.0	19	20
6	5.0	19	20
7	1.0	20	101
7	1.2	20	45
7	2.0	20	25
7	5.0	20	25
8	1.0	17	39
8	1.2	17	18
8	2.0	17	18
8	5.0	17	18

Puzzle	Weight	Solution Cost	Expanded Nodes
9	1.0	50	100,087
9	1.2	50	21,743
9	2.0	50	514
9	5.0	80	281
10	1.0	16	88
10	1.2	16	49
10	2.0	16	27
10	5.0	16	27
11	1.0	51	1,453,717
11	1.2	51	187,001
11	2.0	63	1,493
11	5.0	75	397
12	1.0	17	169
12	1.2	17	70
12	2.0	17	51
12	5.0	17	37

Table 1: Solution cost and expanded nodes for each puzzle, using the Linear Conflict heuristic at four weight settings.

5 Discussion

A few clear patterns emerge from Table 1.

- **Search effort drops sharply as W grows.** For nearly every puzzle, the number of expanded nodes falls by one to three orders of magnitude between $W = 1.0$ and $W = 5.0$. The effect is most dramatic on the harder instances: puzzle 11 goes from over 1.45 million expanded nodes at $W = 1.0$ down to under 400 at $W = 5.0$, and puzzle 9 shows a similar collapse from roughly 100,000 to a few hundred.
- **Optimality is often preserved at moderate weights.** For several puzzles (2, 4, 6, 7, 8, 10, 12) the solution cost at $W = 1.2$ and even $W = 2.0$ matches the $W = 1.0$ (optimal) cost exactly, while still cutting the number of expanded nodes substantially. This suggests that Linear Conflict is already an informative heuristic, so a modest amount of extra greediness rarely forces a longer path.
- **Easy puzzles are insensitive to weight.** Puzzles that are already cheap to solve at $W = 1.0$ (puzzle 6, 7, 8, 10) show almost no change in expanded nodes across weights, since the search space is small enough that greediness has little left to exploit.
- **Large weights sacrifice optimality on harder puzzles.** The puzzles with the largest search trees at $W = 1.0$ (3, 9, 11) are exactly the ones where $W = 5.0$ produces noticeably longer solutions – for example puzzle 11’s cost rises from 51 to 75 moves. This is the expected behaviour of Weighted A*, since a larger weight favours states that look close to the goal even when the true path is longer.

- **$W = 1.2$ is a strong middle ground.** Across the whole set, $W = 1.2$ typically returns the same or nearly the same solution cost as $W = 1.0$ while reducing expanded nodes by roughly 3-5 $\times$ on the harder puzzles, making it the best balance between speed and solution quality in this benchmark.

6 Conclusion

The results confirm the theoretical trade-off underlying Weighted A*: inflating the heuristic with a weight $W > 1$ shrinks the explored search space, often dramatically, at the risk of returning a longer-than-optimal solution. In this benchmark, small weights around 1.2 preserved optimal or near-optimal solutions for most puzzles while still cutting search effort several-fold, whereas a weight of 5.0 minimised search cost but produced clearly suboptimal solutions on the hardest instances. The right setting for W is therefore a function of whether an application values guaranteed optimality or fast, resource-light search.